Day 5 - Strings, Template Literals, JSON, and Dates

Day 5 — Strings, Template Literals, JSON, and Dates

Objectives

- Get comfortable with the string methods you'll use daily
- Use template literals fluently — you've already seen them a few times; today you get the full picture
- Use `JSON.stringify`/`JSON.parse` to convert between JavaScript objects and JSON text
- Get a working, practical understanding of the `Date` object

Strings — the methods that matter

Strings in JavaScript are immutable — exactly like Python strings, every "modifying" method actually returns a brand new string, leaving the original untouched:

```
const s = "  Hello, World!  ";

s.trim();                  // "Hello, World!"          -- removes whitespace from both ends
s.toLowerCase();              // "  hello, world!  "
s.toUpperCase();                 // "  HELLO, WORLD!  "
s.replace("l", "L");                // "  HeLLo, World!  " -- replaces only the FIRST match!
s.replaceAll("l", "L");                // "  HeLLo, WorLd!  " -- replaces EVERY match
s.split(",");                             // ["  Hello", " World!  "] -- splits into an ARRAY where
["a", "b", "c"].join(",");                    // "a,b,c" -- the reverse: glue an array of strings t
s.startsWith("  He");                            // true
s.endsWith("!  ");                                 // true
s.includes("World");                                 // true -- does this string CONTAIN that subst
s.indexOf("World");                                    // 9 -- the index where "World" starts, or -
s.length;                                                 // a PROPERTY, not a method -- no parentl
```

A genuine trap worth knowing about explicitly: `.replace()` with a plain string as its first argument replaces only the *first* match — you need `.replaceAll()` (a more recently added method) to replace every occurrence, or, in older code you might encounter, a special pattern-matching tool called a regular expression with a "global" flag. Always reach for `.replaceAll()` when you mean "every occurrence," rather than assuming `.replace()` does that.

Template literals — the modern way to build strings, using backticks

You've already used these in earlier lessons without a full explanation — a **template literal** is written with backticks (` `) instead of regular quotes, and lets you embed the value of a variable or expression directly inside the text using ${...}`:

```
const name = "Ana";
const score = 91.5;
```

```
console.log(`${name} scored ${score}%`);           // "Ana scored 91.5%"
console.log(`${name} scored ${score.toFixed(1)}%`);    // "Ana scored 91.5%" -- .toFixed(1) rounds t
```

`${...}` can hold any JavaScript expression, not just a plain variable name — `${score.toFixed(1)}`, `${a + b}`, even a function call, all work directly inside the braces.

Template literals also let you write text across multiple lines directly, without any special escape character — something regular quoted strings can't do at all:

```
const multiline = `Line one
Line two
Line three`;
```

Template literals are the modern, preferred way to build any string that includes a variable's value — prefer them over the older style of joining strings with `+` (`"Hello, " + name + "!"`), which is harder to read, especially once several values are being combined.

Number formatting, since `.toFixed()` just came up

```
(3.14159).toFixed(2);       // "3.14" -- rounds to 2 decimal places, returns a STRING (not a number
(1000000).toLocaleString();    // "1,000,000" -- inserts thousands separators, using your system's
```

Note `.toFixed()` returns a *string*, not a number — if you need to do further math with the rounded value, you'd have to convert it back with `Number(...)` first.

JSON — converting between JavaScript objects and portable text

JSON (JavaScript Object Notation) is, fittingly, named after this very language — it's a plain-text format for representing data (objects, arrays, strings, numbers, booleans, null) that's understood by virtually every programming language, and is the standard way programs send structured data to each other over the internet (you'll use this for real on Day 11, talking to a web API).

```
const data = { name: "Ana", tags: ["admin", "active"], active: true };

const jsonText = JSON.stringify(data);                  // '{"name":"Ana","tags":["admin","active"],
const jsonTextPretty = JSON.stringify(data, null, 2);       // same, but nicely indented with 2 spac

const parsedBack = JSON.parse(jsonText);                       // converts the JSON string back into
console.log(parsedBack.name);                                     // "Ana"
```

Naming pattern to remember: `stringify` = JavaScript value → JSON text (turn it INTO a string). `parse` = JSON text → JavaScript value (read a string and turn it back into real data). The middle `null` and `2` arguments to `JSON.stringify` are optional — `null` here just means "no special filtering of which properties to include" (a feature you won't need yet), and `2` sets the indentation width for pretty-printing. If you just want the compact, one-line version, call `JSON.stringify(data)` with no extra arguments at all.

A trap worth knowing, mirroring the Python track's exact same warning if you've seen it: JSON has no concept of `undefined`, functions, or several other JavaScript-specific things. `JSON.stringify` will simply *omit* any object property whose value is `undefined` or a function, entirely silently, without any warning — worth remembering if data seems to be mysteriously missing after a round trip through JSON.

The `Date` object — working with dates and times

```
const now = new Date();           // the current date and time, right now, at the moment this line r
console.log(now);                    // e.g. 2026-07-01T14:32:10.123Z (an ISO-format text representa

const specific = new Date(2026, 0, 15);   // January 15, 2026 -- NOTE: months are 0-indexed! 0 = Jan
console.log(specific.getFullYear());          // 2026
console.log(specific.getMonth());               // 0  -- January, NOT 1! This trips up every JS begi
console.log(specific.getDate());                  // 15 -- the day of the month
console.log(specific.getDay());                     // day of the WEEK (0 = Sunday, 6 = Saturday) --

now.toISOString();      // "2026-07-01T14:32:10.123Z" -- a standard, portable text format, great for
now.getTime();             // a plain number: milliseconds since January 1, 1970 -- useful for compa
```

The single most important gotcha to remember today: months are zero-indexed (January is `0`, December is `11`), while days of the month are NOT (the 1st of the month is `1`, as you'd expect) — this specific inconsistency has confused countless JavaScript developers, at every experience level, and is worth simply memorizing now rather than re-discovering through a bug later.

Comparing dates: since `.getTime()` gives you a plain number (milliseconds), you can subtract two dates to find the time between them:

```
const start = new Date(2026, 0, 1);
const end = new Date(2026, 0, 15);
const millisecondsBetween = end.getTime() - start.getTime();
const daysBetween = millisecondsBetween / (1000 * 60 * 60 * 24);   // convert milliseconds -> second
console.log(daysBetween);   // 14
```

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.